Decision-Model-Guided Function Calling with Cumulative Verification Feedback

Anonymous Author(s)

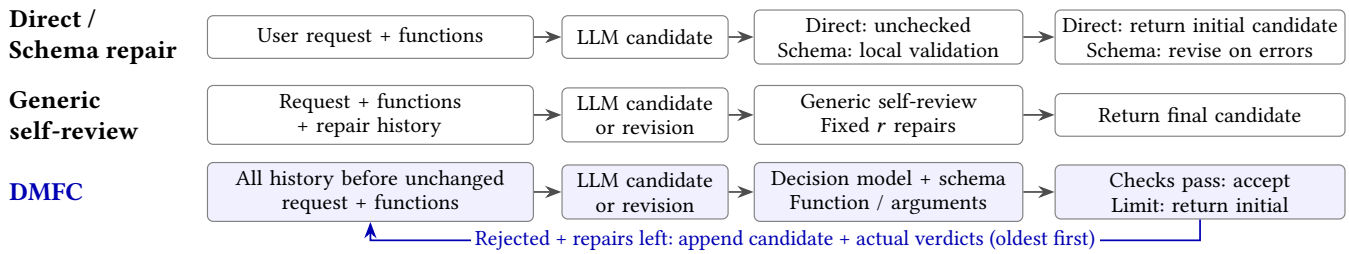


Figure 1: Comparison of evaluated mechanisms (schematic; not measured results). DMFC retains all rejected candidates and actual verdicts before the unchanged request, checks arguments only for a call, and executes no external function during repair.

Abstract

Reliable function calling requires Large Language Models (LLMs) to choose appropriate functions and generate arguments that match user intent. However, even syntactically valid calls can be semantically wrong, for instance by selecting the wrong function or producing mismatched arguments. Prior work uses verification and iterative refinement, but it remains difficult to efficiently carry verification feedback forward across attempts. In this paper, propose **Decision-Model-Guided Function Calling (DMFC)**, an inference-time framework that verifies function selection and argument correctness with separate binary judgments and then refines calls using cumulative feedback from previously rejected candidates without executing external tools. On Berkeley Function Calling Leaderboard (BFCL), DMFC improves accuracy from 64.1% to 74.8% with GPT-5-nano and from 81.1% to 82.1% with GPT-5.4-mini. Retaining verification history further outperforms using only the latest feedback by 1.1 percentage points highlighting the value of cumulative verification for reliable function calling.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**.

Keywords

function calling, decision models, verification, in-context learning

ACM Reference Format:

Anonymous Author(s). 2027. Decision-Model-Guided Function Calling with Cumulative Verification Feedback. In *Proceedings of The ACM Web Conference 2027 (WWW '27).* ACM, New York, NY, USA, 4 pages.

1 Introduction

Function calling enables Large Language Models (LLMs) to interact with external tools and APIs by translating natural-language requests into structured function invocations. Given a user request and a set of available functions, an LLM must decide whether to call a function, select the appropriate one, and generate arguments that reflect the user's intent. This capability is central to LLM-based agentic systems that retrieve information, execute actions, and make decisions in external environments. However, syntactic validity does not guarantee semantic correctness: a call can satisfy a schema while selecting the wrong function or filling arguments with incorrect values.

Recent work improves function-calling reliability through schema validation, tool-use verification, and iterative refinement. Schema-based methods enforce structural constraints, while self-refinement uses model-generated critiques to identify and correct errors. Complementary verification-oriented methods explicitly assess function selection and argument values, and feedback-driven agent frameworks revise subsequent actions using signals such as tool-use error detection or execution observations.

Building on these lines of work, recent advances in decision modeling offer a promising approach to providing targeted judgments over candidate outputs. Unlike conventional LLM-based critiques, decision models can produce structured binary judgments to assess whether specific evaluation criteria are satisfied. Recent approaches further explore confidence-aware verification, where lightweight decision models evaluate candidate outputs and selectively defer uncertain cases to stronger judges, improving evaluation efficiency.

However, turning such judgments into reliable improvements remains challenging. **First**, structural validation alone cannot determine whether a function call aligns with the user's request, particularly when the correct action is to call no function. **Second**, function selection and argument generation are distinct error sources: a model may choose the right function but provide wrong arguments, or generate plausible arguments for the wrong function, which calls for fine-grained verification. **Third**, although iterative refinement can correct mistakes, using only the latest feedback can discard useful signals from earlier failed attempts. These challenges motivate leveraging decision-model verification during refinement while retaining verification history across attempts.

To this end, we propose **Decision-Model-Guided Function Calling (DMFC)**, an inference-time framework that integrates decision-model-based verification with cumulative feedback-driven

WWW '27, Dublin, Ireland
2027.

refinement. DMFC evaluates each generated call with two complementary binary judgments—function selection correctness and argument correctness—in addition to structural schema validation. When a candidate fails verification, the rejected call and its judgments are stored as cumulative feedback and injected into the LLM context to guide subsequent revisions. A call is accepted only when all applicable verification criteria are satisfied, enabling iterative correction without executing external functions.

We evaluate DMFC using diverse decision-model verifier on the Berkeley Function Calling Leaderboard (BFCL) and xxx. DMFC improves accuracy from 64.1% to 74.8% with GPT-5-nano and from 81.1% to 82.1% with GPT-5.4-mini. Retaining verification history further outperforms latest-only feedback by 1.1 percentage points, with the largest gains from preventing unnecessary function calls at the cost of additional inference compute.

Our main contributions are threefold:

- We introduce a decision-model-guided verification framework that explicitly evaluates function selection and argument correctness before execution.
- We propose a cumulative feedback-driven refinement mechanism that leverages historical verification judgments to improve subsequent function calls.
- We conduct comprehensive experiments and ablation studies to evaluate the effectiveness of the proposed framework in terms of function-calling accuracy, correction behavior, and computational efficiency.

2 Methodology

2.1 Task Definition

Let x be a request, $\mathcal{F}$ the available functions, and $c = (f, a)$ a proposed function and arguments, or $c = \varnothing$ for no call. We study a single call-or-no-call decision before execution. Correctly withholding an irrelevant call matters because valid syntax alone cannot prevent an unintended external action. We evaluate one next call, not interactive tool execution; API-Bank inputs may contain several prior dialogue turns.

Toolformer learns when to use APIs [6]. τ-bench evaluates tool-agent interactions under user and policy constraints [10], while API-Bank includes executable API calls in dialogue [2]. Our task isolates a single proposed call before execution. Many-shot learning studies larger demonstration sets [1]; our history instead contains failed attempts at one request.

2.2 Decision-Model Verification

DMFC separates a generator G from a decision model J:

$$c_t = G(H_t, x, \mathcal{F}), \qquad p_f^{(t)} = J_f(x, \mathcal{F}, c_t), \quad p_a^{(t)} = J_a(x, \mathcal{F}, c_t). \tag{1}$$

Here H_t is the rejected-call history, initially empty. The function question checks whether the selected function, including no call, fits x. The argument question checks required values, support in x, and parameter roles; it is skipped for no call. A readable call with missing arguments still receives both checks. Passing the argument check alone cannot establish correct function selection.

A decision model supplies external binary judgments; our evaluated implementation uses TypeSafe's `jev-1.13.0` [8]. Its `Noul`

Algorithm 1 DMFC: verification and cumulative repair

Require: Request x, functions $\mathcal{F}$, generator G, decision model J
$H \leftarrow [\,]$
for $t = 0, \ldots, r$ **do**
  $c_t \leftarrow G(\text{Prefix}(H, x), \mathcal{F})$
  $(s, e, \text{readable}) \leftarrow \text{SchemaCheck}(c_t, \mathcal{F})$
  $p_f, p_a \leftarrow \text{null}, \text{null}$
  if readable **then**
    $p_f, p_a \leftarrow J(x, \mathcal{F}, c_t, \text{ApplicableChecks}(c_t))$
  end if
  if $\text{Accept}(s, c_t, p_f, p_a) = 1$ **then return** c_t
  if $t < r$ **then** $H \leftarrow H \| [(c_t, \text{Feedback}(e, p_f, p_a))]$
end for
return c_0

primitive returns the probability of *yes*, not a separate confidence score [7, 9]. JEV-as-a-Judge studies confidence-based acceptance and escalation [3]; process supervision trains step verifiers for mathematical reasoning [4]. These are adjacent uses of verification, not evidence that other decision models have been tested in DMFC.

2.3 Acceptance and Cumulative Repair

Local validation checks readability, function availability, required arguments, and types; $S(c) = 1$ denotes success. All applicable checks must pass on the same candidate:

$$A_t = S(c_t)\mathbf{1}[p_f^{(t)} \geq \tau_f] \begin{cases} 1, & c_t = \varnothing, \\ \mathbf{1}[p_a^{(t)} \geq \tau_a], & \text{otherwise.} \end{cases} \tag{2}$$

Null required scores fail this gate. After rejection, feedback d_t contains the actual probabilities, pass/fail verdicts, thresholds, and local errors:

$$H_{t+1} = H_t \,\|\, [(c_t, d_t)]. \tag{3}$$

The generator receives all records, oldest first and marked as not executed, before the unchanged original request. The decision model receives only the current request, schemas, and candidate. Readable schema-invalid candidates still receive binary checks; unreadable candidates receive local errors only. Reference answers never enter either model's requests.

A repair replaces the initial answer only after acceptance; if the limit is exhausted, DMFC returns c_0. No external function executes during this loop. Acceptance is a verifier decision, not a guarantee of benchmark correctness.

3 Experiment Setting & Results

3.1 Evaluation and Implementation

We evaluate 3,051 BFCL examples: simple Python (400), multiple (200), irrelevance (240), live simple (258), live multiple (1,053), live irrelevance (884), and live relevance (16). The AST checker compares names and arguments with allowed answers, not execution success. Irrelevance requires no call. Live relevance scores call presence only and is excluded from the 3,035-example main accuracy. Multiple means selecting among available functions; parallel, multi-turn, web-search, memory, and execution tasks are excluded.

Generators are `gpt-5-nano` (2025-08-07), `gpt-5.4-mini` (2026-03-17), and `claude-sonnet-5-5`. GPT uses reasoning effort `none`; Claude receives no thinking override. All use a 4,096-token output cap and automatic, nonparallel native tool choice. The decision model is `jev-1.13.0`; $r = 3$ and $\tau_f = \tau_a = 0.5$. These settings

Benchmark	Generator	Direct (%)	**DMFC** (%)	API s (D / M)	USD/1k (D / M)
BFCL	nano	64.1 ± 0.3	**74.8** ± 0.3	1.48 / 2.81	0.056 / 0.210
BFCL	mini	81.1 ± 0.2	**82.1** ± 0.1	1.30 / 1.98	0.703 / 1.137
BFCL	Sonnet 5.5	**88.3** ± 0.3	87.0 ± 0.3	2.55 / 4.16	4.168 / 6.665
API-Bank†	nano	60.0 ± 2.4	**66.5** ± 1.9	2.38 / 4.46	0.043 / 0.783
API-Bank†	mini	78.2 ± 0.6	**79.5** ± 0.6	1.49 / 2.03	0.814 / 1.460
API-Bank†	Sonnet 5.5	86.3 ± 0.4	**87.2** ± 0.0	1.81 / 2.79	20.827 / 24.580

Table 1: Direct/DMFC accuracy: mean ± sample s.d. of three runs, $r = 3$; bold compares methods per row. API seconds and USD/1k are Direct/DMFC pairs. BFCL: 3,035 accuracy cases, 3,051 time/cost cases; †API-Bank: 156 cases, exploratory adapted exact match. Time includes generation/verification only; cost omits 7 records with unavailable usage/prices. — means unavailable.

Method	Call	Func.	Args.	API s	USD/1k
Direct	63.8	74.8	81.2	1.30	0.056
Schema repair	65.8	76.4	81.8	1.36	0.059
Self-refine	70.0	80.3	82.2	5.47	0.265
DMFC-F	73.2	84.5	81.4	2.24	0.172
DMFC-A	72.5	82.6	**82.7**	2.21	0.169
DMFC-L	73.5	84.3	82.2	2.60	0.203
DMFC	**74.6**	**85.3**	82.1	2.61	0.211

Table 2: Ablation: one `gpt-5-nano` run, $r = 3$. Call/function accuracy (%) excludes live relevance. Arguments are scored only when the function is correct; bold marks each accuracy column's maximum.

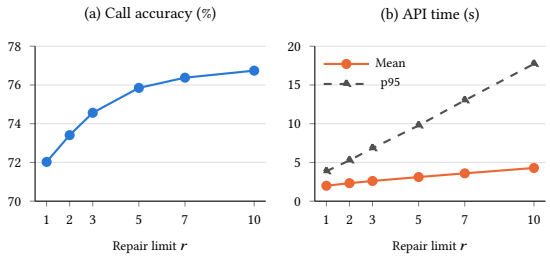


Figure 2: DMFC with `gpt-5-nano`: accuracy and API time over 3,051 examples from one run continued to $r = 10$ and cut at each limit. At $r = 3$, values match Table 2. API time sums generation and verification durations, excluding retry waits and tool execution.

were chosen on the evaluated examples after thresholds 0.8/0.7 and last-candidate fallback; there is no held-out tuning set. Filtered or truncated generations count as incorrect.

For transfer, we adapt API-Bank [2] to first-call prediction: retain dialogue prefixes before their first API turn and supply a fixed 53-API catalog, independent of gold calls. Eight gold/schema conflicts are excluded before inference, leaving 204 cases grouped into development (48) and test (156). A common user instruction requests a native call candidate without execution. We score API names and type-normalized arguments; this exact-match adaptation differs from API-Bank's native execution score. The BFCL settings transfer unchanged. Four development cases verify the interface, without tuning thresholds or fallback; the prompt was revised after inspecting earlier test outcomes, so this evaluation is exploratory.

Compared methods. All methods share initial candidates. *Direct* returns the initial call; *Schema repair* uses local errors; *Self-refine* uses the generator's generic self-review for all r repairs, an adaptation rather than a reproduction of Self-Refine [5]. **DMFC-F** uses only function feedback and its acceptance gate; **DMFC-A** uses only argument feedback and its gate. Both compute applicable judgments but expose and require only their designated judgment, plus schema validation; DMFC-A accepts no call on schema alone. **DMFC-L** retains only the latest rejected candidate and feedback. Full **DMFC** retains all records. F/A isolate the two checks; L isolates history retention with the same thresholds, budget, and initial calls.

3.2 Main Experiment

Table 1 shows BFCL gains of 10.7 points for nano (paired 95% interval 9.7–11.6) and 1.0 for mini (0.5–1.5). Nano's largest category gain is on Irrelevance, 27.2% to 78.2%, driven by withholding unnecessary calls. Claude instead loses 1.3 points (interval −1.8 to −0.7): across three runs, repair fixes 58 initial errors but introduces 174 after accepted revisions. Its mean API time rises from 2.55 to 4.16 s.

API-Bank gains are 6.4 points for nano (interval 3.8–9.4), 1.3 for mini (0.2–2.6), and 0.9 for Claude (0.0–2.6). Claude's interval includes zero. These mixed effects do not establish a uniform benefit across generators or end-to-end tool-use success.

3.3 Ablation and Latency

In Table 2, both checks outperform either alone on call accuracy. Full history beats latest-only feedback by 1.1 points (95% interval 0.4–1.7), with similar mean API time (2.61 vs. 2.60 s). DMFC-A's higher conditional argument score does not imply better overall calls because the scored subsets differ.

Figure 2 cuts one continued trajectory at each repair limit; these are not independent runs. Increasing r from 3 to 10 raises accuracy from 74.6% to 76.7%, mean API time from 2.61 to 4.29 s, and p95 from 6.83 to 17.72 s; cost grows from \$0.211 to \$0.399 per 1,000 requests. The smaller accuracy gains therefore come with a larger latency tail.

Fallback and measurement limits. API time sums generation and verification, excluding retry waits, scoring, and tool execution. Cost includes initial and repair generation plus verification; unknown usage is omitted from known-cost means. Failed unsaved repair work is excluded, so costs are not invoice totals. Intervals bootstrap example IDs 2,000 times (seed 1729), averaging repetitions

per ID. BFCL settings and API-Bank's input amendment followed inspected outcomes. API-Bank has only 156 test cases and no validated no-call negatives. Three generators, one verifier, single-run ablations, English-dominated public data, and ten retained BFCL annotation errors limit generalization. Native schema rejections count as incorrect. Claude BFCL concurrency changed from 8 to 32 with prefixes preserved; API times do not establish provider speed rankings. Acceptance and correctness remain separate.

4 Conclusion

DMFC combines function and argument judgments with cumulative rejected-call feedback before execution. DMFC improves the GPT comparisons but reduces Claude accuracy on BFCL, while adding API time and cost. The exploratory API-Bank first-call evaluation extends the evidence beyond BFCL but does not measure execution success. Independent evaluation, other decision-model families, and open-weight generators remain necessary.

References

[1] Rishabh Agarwal, Avi Singh, Lei Zhang, Bernd Bohnet, Luis Rosias, Stephanie Chan, Biao Zhang, Ankesh Anand, Zaheer Abbas, Azade Nova, John D. Co-Reyes, Eric Chu, Feryal Behbahani, Aleksandra Faust, and Hugo Larochelle. 2024. Many-Shot In-Context Learning. In *Advances in Neural Information Processing Systems*, Vol. 37. 76930–76966. doi:10.52202/079017-2447

[2] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3102–3116. doi:10.18653/v1/2023.emnlp-main.187

[3] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. 2026. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. *arXiv preprint arXiv:2609.26550* (2026). Version 5, October 9, 2026. arXiv:2609.26550 [cs.AI] doi:10.48550/arXiv.2609.26550

[4] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *International Conference on Learning Representations*. https://arxiv.org/abs/2305.20050

[5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. 46534–46594. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. 68539–68551. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[7] TypeSafe. 2026. Confidence. Technical documentation. Accessed October 17, 2026. https://docs.typesafe.ai/confidence

[8] TypeSafe. 2026. Models: Jev and the System One API. Technical documentation. Accessed October 15, 2026. https://docs.typesafe.ai/models

[9] TypeSafe. 2026. `Noul`. Technical documentation. Accessed October 17, 2026. https://docs.typesafe.ai/primitives/noul

[10] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045* (2024). https://arxiv.org/abs/2406.12045